Data Acquisition for NLP: Course Notes

1. Why Data Acquisition Comes First

Every natural language processing system starts with text, and that text rarely arrives in a clean, ready-to-use form. Customer emails, scanned contracts, news articles, product reviews and research papers are all stored in different formats, and each format hides the words behind its own layer of structure.

A model can only be as good as the data it is trained on. If the acquisition step silently drops pages, merges columns or keeps navigation menus as if they were content, every later step - tokenization, tagging, classification - inherits those mistakes. For this reason acquisition is treated as an engineering task of its own, not as a quick preliminary.

In practice a data scientist spends a surprising share of a project on this stage. Surveys of industry teams often describe data preparation as the most time-consuming part of building a model, and the first half of that work is simply getting the text out of where it lives.

2. Text Inside PDF Files

A PDF page is a set of drawing instructions. Each glyph is placed at an x and y position with a font, and the file has no native idea of a word, a sentence or a paragraph. Extraction libraries rebuild the reading order by walking the content stream and grouping glyphs that sit close together.

This reconstruction works well for single-column reports like this one, but it struggles with two-column papers, tables and footnotes. Lines from the left and right columns can be interleaved, and headers or page numbers appear in the middle of the extracted text.

Scanned documents are a separate problem. A scanned page is an image, so a text extractor returns an empty string - the words only exist as pixels. Recovering them needs optical character recognition, which is outside the scope of a plain PDF parser.

Because of these issues, extracted PDF text should always be treated as “raw” input. Removing stray line breaks, re-joining hyphenated words and filtering repeated headers are routine follow-up steps.

3. Word Documents and Their Structure

A .docx file is a ZIP archive of XML parts. Each paragraph is its own XML node, and each node carries a style name such as Heading 1 or Normal. That style name is what lets a program separate headings from body text without guessing from font size.

The approach has a weakness: authors who make a line bold and large instead of applying a heading style produce a document that looks structured but is not. A parser that trusts style names will classify those lines as ordinary body text.

4. JSON APIs

Many organisations publish data through web APIs that answer an HTTP request with JSON. A JSON object maps directly to a Python dictionary and a JSON array maps to a list, so parsing is deterministic and needs no custom logic.

APIs are designed for machines. Their field names are documented and stable, which makes them far less fragile than scraping. When an API exists for a dataset it should be the first choice - it is faster, cleaner and usually allowed by the provider’s terms of use.

Real APIs return results in pages. A request might return the first thirty records together with a total count, and the client must ask for the remaining pages using parameters such as limit and skip. Forgetting this step is a common reason for datasets that are smaller than expected.

5. Scraping HTML Pages

HTML is a tree of nested tags. Elements such as a book title or a quote are usually marked up with the same tag and class on every repetition, which is what makes structured scraping possible.

Tools like BeautifulSoup search this tree by tag name and attribute rather than by scanning raw text with regular expressions. Tree-based extraction keeps working when spacing or attribute order changes, whereas a regular expression breaks silently.

Responsible scraping means reading the site’s robots.txt, sending a clear user-agent, adding a delay between requests and never collecting personal data without permission. A scraper that floods a small server is not just rude - it can be blocked or cause real harm.

6. Cleaning the Acquired Text

Once text has been extracted, a short cleaning pass makes it usable: lower-casing, removing punctuation, normalising whitespace and fixing encoding errors. The exact steps depend on the task. Sentiment analysis may keep exclamation marks, while topic modelling usually removes them.

Counting word frequencies right after cleaning is a quick sanity check. If the most frequent tokens are page numbers or fragments of a running header, the extraction step needs another look before any model is trained on the data.

Good acquisition code is also repeatable. Saving the raw text alongside the cleaned version, recording the source URL and the date of collection, and keeping the scripts under version control make it possible to rebuild the corpus later.